



**INSTITUTO SUPERIOR POLITÉCNICO DE TECNOLOGIAS E
CIÊNCIAS
DEPARTAMENTO DE ENGENHARIAS E TECNOLOGIAS
CURSO DE ENGENHARIA INFORMÁTICA**

SISTEMA DE GESTÃO DE BIBLIOTECA

Autores:

Daniel Filipe – 20241123

Jurelmo Botelho – 20241297

Kenny Marques – 20242411

**LUANDA
2026**

ÍNDICE

1. INTRODUÇÃO	1
2. METODOLOGIA.....	2
2.1. Bibliotecas Usadas.....	2
2.2. Modularização Do Projecto	2
2.3. Tipos Abstratos de Dados (TADs) E Outras Estruturas Utilizadas.....	4
2.3.1. AVLNode	4
2.3.2. Book	4
2.3.3. User.....	4
2.3.4. Loan	5
2.3.5. ReportData.....	5
2.3.6. Session	6
2.3.7. Date.....	6
2.3.8. Role.....	7
2.3.9. LoanStatus	7
2.4. Principais Funcionalidades do Sistema	7
2.4.1. Autenticação	7
2.4.2. Gestão de Livros	7
2.4.3. Gestão de Leitores	8
2.4.4. Gestão de Empréstimos	8
2.4.5. Gestão de Reservas	8
2.4.6. Geração de Relatórios.....	8
2.5. Regras de Negócio.....	8
2.6. Divisão Das Tarefas Entre Os Membros	9
CONCLUSÃO.....	10
REFERÊNCIAS BIBLIOGRÁFICAS	11

1. INTRODUÇÃO

As bibliotecas desempenham um papel fundamental na organização e disponibilização de recursos de informação. O presente projeto consiste no desenvolvimento de um Sistema de Gestão de Biblioteca, implementado na linguagem C, cujo principal objetivo é facilitar a administração dos recursos da biblioteca através de uma solução organizada, eficiente e de fácil utilização. O sistema permite gerir livros e leitores, efetuar empréstimos e devoluções, controlar reservas de livros indisponíveis, consultar informação relevante e gerar relatórios estatísticos sobre a atividade da biblioteca.

Para garantir um desempenho eficiente, o projeto faz uso de diferentes estruturas de dados, destacando-se a utilização de uma Árvore AVL genérica como estrutura principal para o armazenamento e organização dos livros e dos leitores.

2. METODOLOGIA

2.1. Bibliotecas Usadas

Para a realização desse projecto foram utilizadas as seguintes bibliotecas:

- **<stdio.h>**: para entrada e saída padrão, essencial para interação com o utilizador e geração de relatórios;
- **<stdlib.h>**: para gestão de memória e utilitários, indispensável para estruturas dinâmicas como listas ligadas;
- **<string.h>**: para manipulação de strings e trabalhar corretamente com nomes e textos;
- **<locale.h>**: para definição de localização linguística, garante correta apresentação de acentos e caracteres especiais em português;
- **<ctype.h>**: disponibiliza funções para classificação e manipulação de caracteres.

2.2. Modularização Do Projecto

Com o objetivo de tornar o código mais organizado e de fácil manutenção, o sistema foi dividido em vários módulos, sendo cada um responsável por uma funcionalidade específica. Cada módulo é composto, sempre que aplicável, por um ficheiro de cabeçalho (.h), onde são declaradas as estruturas de dados, constantes e protótipos das funções, e por um ficheiro de implementação (.c), onde se encontra o código correspondente. Sendo a divisão do projeto a seguinte:

- **types.h**: este módulo reúne as definições globais utilizadas por todo o sistema. Contém constantes, limites máximos de tamanho para cadeias de caracteres, valores relacionados com empréstimos, tipos enumerados (como os papéis dos utilizadores e o estado dos empréstimos) e declarações antecipadas (forward declarations) das principais estruturas do projeto. O seu objetivo é centralizar definições comuns e evitar dependências circulares entre módulos.
- **auth.h / auth.c**: responsável pelo sistema de autenticação dos utilizadores. Implementa as funcionalidades de criação de contas, validação das credenciais de acesso, gestão de sessões e outras operações relacionadas com o controlo de acesso ao sistema.
- **avl.h / avl.c**: implementa a Árvore AVL genérica utilizada como principal estrutura de armazenamento do sistema. O módulo disponibiliza operações de inserção, remoção, pesquisa, balanceamento, rotações e percursos da árvore, permitindo reutilizar a mesma implementação para diferentes tipos de dados.

- **user.h / user.c:** responsável pela gestão dos leitores da biblioteca. Contém as estruturas e funções necessárias para criar, atualizar, remover, pesquisar e consultar utilizadores, bem como gerir os empréstimos associados a cada leitor.
- **book.h / book.c:** implementa a gestão dos livros existentes na biblioteca. Inclui funções para registo, atualização, remoção, pesquisa e consulta de livros, além da gestão da disponibilidade de exemplares e das filas de reservas.
- **date.h / date.c:** disponibiliza funções relacionadas com a manipulação de datas, incluindo obtenção da data atual, cálculo de prazos de devolução, comparação entre datas e verificação de atrasos nos empréstimos.
- **ui.h / ui.c:** responsável pela interface textual do sistema. Implementa os menus, apresentação de informações ao utilizador e interação através da consola, separando a lógica da aplicação da camada de apresentação.
- **loan.h / loan.c:** implementa todas as funcionalidades relacionadas com os empréstimos e devoluções de livros. Gere a criação e atualização dos empréstimos, o controlo do estado dos mesmos e a ligação entre livros e leitores.
- **reserve.h / reserve.c:** implementa a estrutura de fila utilizada para a gestão das reservas de livros. O módulo disponibiliza operações de inserção, remoção e consulta de elementos, respeitando o princípio FIFO (First In, First Out).
- **data_persistence.h / data_persistence.h.c:** responsável pela persistência dos dados do sistema. Implementa funções para guardar e carregar informação em ficheiros, permitindo manter os dados da biblioteca entre diferentes execuções da aplicação.
- **report.h / report.c:** implementa a geração de relatórios estatísticos sobre a atividade da biblioteca, incluindo informações relativas a livros, leitores, empréstimos, devoluções e reservas, auxiliando na análise da utilização do sistema.
- **validation.h / validation.c:** implementa as funções de validação dos dados introduzidos pelos utilizadores. Este módulo verifica a integridade das informações, como formatos de entrada, restrições de tamanho, unicidade de identificadores e cumprimento das regras de negócio, garantindo a consistência dos dados armazenados.

2.3. Tipos Abstratos de Dados (TADs) E Outras Estruturas Utilizadas

Para o desenvolvimento do Sistema de Gestão de Biblioteca utilizou-se vários Tipos Abstratos de Dados (TADs), cada um concebido para representar uma entidade ou estrutura lógica do sistema.

2.3.1. AVLNode

Representa um nó da Árvore AVL, utilizada como principal estrutura de armazenamento dos livros e dos leitores. Cada nó possui uma chave inteira (*key*), um ponteiro genérico para os dados armazenados (*void *data*), a altura do nó e ponteiros para os filhos esquerdo e direito. A utilização de um ponteiro genérico permite que a mesma implementação da árvore seja reutilizada para diferentes tipos de dados.

```
typedef struct AVLNode
{
    int key;
    void *data;
    int height;
    struct AVLNode *left;
    struct AVLNode *right;
} AVLNode;
```

Figura 1-TAD AVLNode

2.3.2. Book

Representa um livro existente na biblioteca. Armazena toda a informação bibliográfica necessária, incluindo identificador, título, autor, editora, categoria, ano de publicação, idade mínima recomendada, quantidade total e quantidade disponível de exemplares. Contém ainda um contador do número de empréstimos realizados e uma fila de reservas associada ao livro.

```
typedef struct Book{
    int id;
    char title[MAX_TITLE];
    char author[MAX_AUTHOR];
    char publisher[MAX_PUBLISHER];
    char category[MAX_CATEGORY];
    int year;
    int minAge;
    int totalQuantity;
    int availableQuantity;
    int timesBorrowed;
    ReserveQueue *reservations;
} Book;
```

Figura 2-TAD Book

2.3.3. User

Representa um utilizador (leitor ou bibliotecário) do sistema. Contém dados pessoais e de autenticação, como identificador, nome, username, palavra-passe, idade e contacto telefónico. Inclui ainda o papel do utilizador (administrador ou estudante), o número de

empréstimos ativos e uma lista ligada com os empréstimos atualmente associados ao leitor.

```
typedef struct User
{
    int id;
    char username[MAX_USERNAME];
    char password[MAX_PASSWORD];
    char name[MAX_NAME];
    int age;
    char phone[MAX_PHONE];

    Role role;

    int activeLoans;
    LoanList myLoans;
} User;
```

Figura 3-TAD User

2.3.4. Loan

Representa um empréstimo de um livro. Cada empréstimo associa diretamente um livro e um utilizador através de ponteiros, evitando duplicação de informação. Além disso, armazena as datas de empréstimo, devolução prevista e devolução efetiva, bem como o estado atual do empréstimo. A estrutura contém ainda dois ponteiros que permitem integrar simultaneamente o empréstimo na lista global de empréstimos e na lista de empréstimos do respetivo utilizador.

```
struct Loan {
    int id;
    Book *book;
    User *leitor;

    Date DataLoan;
    Date DateExpected;
    Date DateReturn;
    LoanStatus status;

    struct Loan *nextGlobal;
    struct Loan *nextUser;
};
```

Figura 4-Loan

2.3.5. ReportData

Estrutura utilizada para armazenar os dados necessários à geração de relatórios estatísticos. Contém vetores de ponteiros para livros e utilizadores organizados segundo diferentes critérios (livros mais requisitados, leitores mais ativos, livros disponíveis, empréstimos em atraso, entre outros), bem como indicadores globais relativos ao funcionamento da biblioteca, como o número total de livros, leitores, empréstimos, devoluções e reservas.

```
typedef struct {
    Book **mostBorrowed;
    int mostBorrowedCount;
    Book **leastBorrowed;
    int leastBorrowedCount;
    Book **neverBorrowed;
    int neverBorrowedCount;
    Book **available;
    int availableCount;
    Book **unavailable;
    int unavailableCount;

    User **mostActive;
    int mostActiveCount;
    User **mostActiveLoans;
    int mostActiveLoansCount;
    User **noLoans;
    int noLoansCount;
    User **overdue;
    int overdueCount;

    int totalBooks;
    int totalUsers;
    int totalLoans;
    int totalReturns;
    int totalReservations;
} ReportData;
```

Figura 5-TAD ReportData

2.3.6. Session

Representa a sessão atual do sistema. Armazena um ponteiro para o utilizador autenticado e um indicador que permite saber se existe uma sessão iniciada. Esta estrutura facilita o controlo do acesso às funcionalidades de acordo com o tipo de utilizador autenticado.

```
typedef struct Session
{
    User *currentUser;
    int isLoggedIn;
} Session;
```

Figura 6-TAD Session

2.3.7. Date

Representa uma data do calendário através dos campos dia, mês e ano. É utilizada em todas as operações relacionadas com empréstimos, devoluções, cálculo de prazos e verificação de atrasos.

```
typedef struct Date {
    int day;
    int month;
    int year;
} Date;
```

Figura 7-TAD Date

2.3.8. Role

Tipo enumerado que identifica o perfil de cada utilizador do sistema. Os valores possíveis são **ROLE_ADMIN**, correspondente ao bibliotecário com permissões de administração, e **ROLE_STUDENT**, correspondente ao leitor comum da biblioteca.

```
typedef enum
{
    ROLE_ADMIN = 1,
    ROLE_STUDENT = 2
} Role;
```

Figura 8-Role

2.3.9. LoanStatus

Tipo enumerado que representa o estado de um empréstimo. Os estados possíveis são **LOAN_ACTIVE** (empréstimo ativo), **LOAN_RETURNED** (empréstimo devolvido) e **LOAN_OVERDUE** (empréstimo em atraso). A utilização desta enumeração facilita o controlo do ciclo de vida de cada empréstimo e melhora a legibilidade do código.

```
typedef enum
{
    LOAN_ACTIVE = 0,
    LOAN_RETURNED = 1,
    LOAN_OVERDUE = 2
} LoanStatus;
```

Figura 9-LoanStatus

2.4. Principais Funcionalidades do Sistema

2.4.1. Autenticação

O sistema possui um mecanismo de autenticação que permite controlar o acesso às diferentes funcionalidades de acordo com o tipo de utilizador.

As funcionalidades disponibilizadas incluem:

- Criação de contas de utilizador;
- Autenticação através de *username* e palavra-passe;

2.4.2. Gestão de Livros

Este módulo é responsável pela administração do acervo bibliográfico da biblioteca. As principais funcionalidades são:

- Registar novos livros;
- Atualizar os dados de livros existentes;
- Remover livros (desde que não possuam empréstimos ativos);
- Consultar todos os livros registados;
- Pesquisar livros.

2.4.3. Gestão de Leitores

Permite administrar todos os utilizadores registados na biblioteca. Entre as funcionalidades disponíveis encontram-se:

- Registo de novos leitores;
- Atualização dos dados pessoais;
- Remoção de leitores;
- Consulta de leitores registados;
- Pesquisa de leitores através do respetivo identificador.

2.4.4. Gestão de Empréstimos

O sistema permite a gestão de empréstimos de livros da biblioteca. As funcionalidades implementadas incluem:

- Efetuar novos empréstimos;
- Registrar devoluções;
- Consultar empréstimos ativos;
- Consultar o histórico de empréstimos.

2.4.5. Gestão de Reservas

Quando um livro não possui exemplares disponíveis, o sistema permite ao utilizador efetuar uma reserva. As principais funcionalidades são:

- Inserção de leitores na fila de reservas;
- Gestão automática da ordem de atendimento segundo a política FIFO (*First In, First Out*).

2.4.6. Geração de Relatórios

O sistema disponibiliza um conjunto de relatórios estatísticos destinados a apoiar a gestão da biblioteca. Entre os relatórios implementados destacam-se:

- Relatórios de Livros
- Relatórios de Leitores
- Relatórios Gerais

2.5. Regras de Negócio

Afim de manter uma boa gestão e o bom funcionamento do sistema, estabelecemos as seguintes regras:

- **Para Livros:** O ID deve ser único. Não é permitido remover um livro que possua empréstimos ativos. A quantidade disponível nunca pode ser superior à quantidade total. A quantidade total não pode ser inferior ao número de exemplares emprestados.

- **Para Leitores:** O ID deve ser único. O Username deve ser único. Não é permitido remover um leitor com empréstimos ativos.
- **Para Empréstimos:** Cada leitor pode possuir no máximo **3 livros** emprestados. Apenas podem ser emprestados livros disponíveis. O sistema deverá verificar a idade mínima exigida pelo livro. Caso não existam exemplares disponíveis, o utilizador poderá entrar na fila de reservas. A devolução atualiza automaticamente a disponibilidade do livro.
- **Para Reservas:** Um leitor não pode reservar um livro que já possui emprestado. Um leitor não pode reservar o mesmo livro mais do que uma vez. A reserva respeita sempre a ordem FIFO.

2.6. Divisão Das Tarefas Entre Os Membros

As tarefas foram divididas da seguinte forma:

- **Daniel Filipe:** redigir o relatório;
- **Jurelmo Botelho:**
- **Kenny Marques:**

CONCLUSÃO

O desenvolvimento do Sistema de Gestão de Biblioteca consolidou de forma prática os conceitos de Programação Estruturada e Estruturas de Dados Avançadas, resultando numa solução robusta e escalável.

Como estudantes de programação, muitas vezes temos a percepção de que a parte do código é a mais importante, mas, com este projeto e também com a disciplina de Estrutura de Dados neste semestre, começámos a compreender e a ter uma outra visão sobre como programar, partindo da organização e definição de parâmetros mais importantes antes mesmo do código.

No início do projeto, uma das coisas mais difíceis foi começar, pois não sabíamos a melhor forma de desenhar aquilo que tínhamos em mente, mas, estudando e praticando mais sobre as estruturas de dados, as arquiteturas de software e padrões de organização, conseguimos desbloquear e avançar com o projeto.

Durante o projeto, aprendemos muitas coisas, realçando aqui a escolha estratégica de uma Árvore AVL genérica como a espinha dorsal do projeto, o que provou ser ideal, garantindo operações de inserção, remoção e pesquisa de livros e utilizadores em tempo logarítmico, mesmo sob grandes volumes de dados. A forma como cada estrutura de dados foi projetada para atuar de forma eficiente em soluções específicas e a arquitetura modular aplicada ao código separaram com clareza as camadas de lógica de negócio (como a validação de regras, gestão de filas FIFO para reservas e manipulação de datas) e a interface textual de utilizador. Esta abordagem facilita futuras manutenções e o reaproveitamento de componentes. Em suma, acreditamos que o projeto atingiu os objetivos pedagógicos e técnicos delineados pois o sistema entrega um ambiente seguro, persistente e bem otimizado.

REFERÊNCIAS BIBLIOGRÁFICAS

AHO, Alfred V.; HOPCROFT, John E.; ULLMAN, Jeffrey D. Data Structures and Algorithms. 1. ed. Reading: Addison-Wesley, 1983.

ANTONIO, Sílvia. Árvores AVL balanceadas e listas ligadas. Luanda: ISPTEC, 2026. Notas de aula da disciplina de Estrutura de Dados.

ÁRVORES Balanceadas. Google, 2026. 1 playlist de vídeos (YouTube). Disponível em: https://www.youtube.com/playlist?list=PLqJK4Oyr5WSgUXgpurvc0TBQx_byFHAIJ. Acesso em: 27 de Junho de 2026.

TUNGA, Emanuel. Introdução à manipulação de ficheiros em C. Luanda: ISPTEC, 2025. Notas de aula da disciplina de Programação I.